

UNINTER
CENTRO UNIVERSITÁRIO INTERNACIONAL

CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

TARLAN MARCOS DALLA VECCHIA

RU: 4896093

PROJETO MULTIDISCIPLINAR TRILHA BACK-END
REDE "RAÍZES DO NORDESTE"

CAPANEMA - PR

2026

SUMÁRIO

1	INTRODUÇÃO.....	3
2	ANÁLISE E REQUISITOS	4
2.1	Requisitos Funcionais	4
2.2	Requisitos Não Funcionais.....	5
2.3	Regras de Negócio.....	6
2.4	Priorização e Justificativa	7
3	MODELAGEM E ARQUITETURA	8
3.1	Diagrama de Casos de Uso.....	8
3.2	Diagrama de Classes.....	8
3.3	Diagrama Entidade-Relacionamento (DER).....	9
3.4	Arquitetura em Camadas.....	10
3.5	Principais Endpoints da API.....	11
3.6	Padrão de Erro	12
3.7	Decisões Técnicas.....	12
4	ENTREGA TÉCNICA	13
4.1	Repositório do Projeto	13
4.2	Como Executar a API	13
4.2.1	Clone o repositório:.....	13
4.2.2	Crie e ative o ambiente virtual:.....	13
4.2.3	Instale as dependências:	13
4.2.4	Execute a API:	13
4.2.5	Acesse a documentação Swagger:.....	13
4.3	Endpoints Implementados	14
4.4	Evidências de Execução.....	14
4.5	Coleção Postman	18
5	PLANO DE TESTES E EVIDÊNCIAS	19
5.1	Estratégia de Testes	19
5.2	Cenários de Teste.....	19
5.3	Evidências	20
5.4	Cobertura dos Testes.....	20
6	CONCLUSÃO	21
7	REFERÊNCIAS.....	22
8	DECLARAÇÃO DE USO DE IA	23
9	INFORMAÇÕES DE ENTREGA.....	24

1 INTRODUÇÃO

O projeto foi desenvolvido para a disciplina de Projeto: Desenvolvimento Back-end, do curso de CST em Análise e Desenvolvimento de Sistemas. O tema tratado foi a rede de lanchonetes "Raízes do Nordeste", apresentada no estudo de caso da disciplina.

A Raízes do Nordeste começou como um negócio familiar em Recife, e hoje já tem unidades em várias capitais e no interior do Brasil. Com o crescimento, a estrutura atual do sistema não consegue mais acompanhar a demanda.

O maior desafio é atender vários canais ao mesmo tempo. O cliente pode fazer pedido pelo aplicativo, por totens nas lojas, no balcão ou para retirada. Cada unidade tem seu próprio cardápio, seu próprio estoque e até regras diferentes de funcionamento. Além disso, existe a exigência de conformidade com a LGPD e a necessidade de integrar com serviços externos de pagamento.

Outro ponto crítico é a disponibilidade. Nos horários de pico, o sistema não pode falhar. Qualquer instabilidade afeta diretamente o faturamento e a experiência do cliente.

Diante desse cenário, o objetivo deste trabalho foi projetar uma solução de Back-end que atendesse aos requisitos principais da rede. Optei por construir uma API REST usando Python com FastAPI, banco de dados SQLite com SQLAlchemy, autenticação via JWT e documentação automática pelo Swagger.

Os principais usuários do sistema são:

- Cliente: faz pedidos, acompanha o status e acumula pontos de fidelidade;
- Atendente: registra pedidos no balcão e consulta o cardápio;
- Cozinha: acompanha o pedido e atualiza o status;
- Gerente/Administrador: gerencia produtos, estoque e emite relatórios para a matriz;
- Sistema de Pagamento (externo): processa os pagamentos solicitados.

O sistema foi pensado para permitir que a rede cresça de forma organizada. A ideia é manter a identidade regional da marca, mas com padronização, segurança de dados e qualidade no atendimento em qualquer canal.

O resultado é uma API funcional, testável e documentada, que pode servir de base para qualquer interface de Front-end.

2 ANÁLISE E REQUISITOS

Nesta seção são apresentados os requisitos funcionais e não funcionais do sistema, bem como as regras de negócio identificadas a partir do estudo de caso. Também é descrita a priorização adotada para o desenvolvimento.

2.1 Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades do sistema, e foram levantados a partir do estudo de caso da rede Raízes do Nordeste.

ID	Descrição	Prioridade
RF01	O sistema deve permitir cadastro de usuários com perfis (ADMIN, GERENTE, CLIENTE).	Alta
RF02	O sistema deve permitir autenticação via login com geração de token JWT.	Alta
RF03	O sistema deve permitir cadastro e listagem de lojas da rede.	Alta
RF04	O sistema deve permitir cadastro de produtos vinculados a uma loja.	Alta
RF05	O sistema deve permitir consulta do cardápio por unidade.	Alta
RF06	O sistema deve permitir criação de pedidos com canal obrigatório (APP, TOTEM, BALCAO, PICKUP, WEB).	Alta
RF07	O sistema deve permitir consulta de pedidos por ID e listagem com filtro por canal.	Alta
RF08	O sistema deve permitir atualização do status do pedido.	Alta
RF09	O sistema deve simular processamento de pagamento (mock), com retorno de aprovação ou negação.	Alta
RF10	O sistema deve atualizar automaticamente o status do pedido após resposta do pagamento.	Alta

RF11	O sistema deve registrar consentimento do usuário para uso de dados (LGPD).	Média
RF12	O sistema deve permitir controle de estoque por unidade.	Média
RF13	O sistema deve permitir cadastro de produtos sazonais.	Baixa
RF14	O sistema deve permitir acúmulo de pontos de fidelidade por cliente.	Média
RF15	O sistema deve permitir emissão de relatórios consolidados para a matriz.	Média

2.2 Requisitos Não Funcionais

Os requisitos não funcionais descrevem características de qualidade, restrições e padrões que o sistema deve atender.

ID	Descrição	Categoria
RNF01	O sistema deve ser implementado como uma API REST, com endpoints padronizados.	Arquitetura
RNF02	A documentação da API deve ser gerada automaticamente via Swagger/OpenAPI.	Documentação
RNF03	As senhas dos usuários devem ser armazenadas com hash bcrypt.	Segurança
RNF04	A autenticação deve ser feita via token JWT com tempo de expiração.	Segurança
RNF05	O sistema deve implementar controle de acesso por perfis (RBAC).	Segurança
RNF06	Endpoints sensíveis devem exigir autenticação.	Segurança
RNF07	O sistema deve estar em conformidade com a LGPD.	Privacidade

RNF08	O sistema deve ser tolerante a falhas na integração com pagamento.	Confiabilidade
RNF09	O sistema deve ter alta disponibilidade nos horários de pico.	Desempenho
RNF10	O sistema deve ser escalável para suportar o crescimento da rede.	Escalabilidade
RNF11	O sistema deve padronizar respostas de erro em formato JSON.	Padronização
RNF12	O sistema deve usar códigos HTTP coerentes.	Padronização
RNF13	O banco de dados deve garantir consistência e integridade referencial.	Persistência
RNF14	O código deve ser organizado em camadas.	Manutenibilidade

2.3 Regras de Negócio

As regras de negócio descrevem as restrições e condições que o sistema deve respeitar para garantir a coerência das operações. Foram identificadas a partir do estudo de caso, com foco no fluxo de pedidos, pagamento e múltiplos canais de atendimento.

ID	Descrição
RN01	Todo pedido deve ter um canal de origem obrigatório (canal_pedido).
RN02	Um pedido só pode ser pago se estiver com status AGUARDANDO_PAGAMENTO.
RN03	Um pedido já pago não pode ser pago novamente (retorna 409 Conflict).
RN04	Pagamentos com valor até R\$ 100,00 são aprovados (regra do mock).
RN05	Pagamentos negados alteram o status do pedido para CANCELADO.
RN06	Pagamentos aprovados alteram o status do pedido para PAGO.

RN07	Canais válidos: APP, TOTEM, BALCAO, PICKUP, WEB.
RN08	Um produto não pode ser cadastrado em loja inexistente.
RN09	Um pedido não pode ser criado para loja inexistente.
RN10	Consentimento LGPD é obrigatório no cadastro.

2.4 Priorização e Justificativa

O critério adotado para o desenvolvimento deste projeto foi fechar o fluxo crítico de ponta a ponta, desde o pedido até a atualização do status após o pagamento. Essa era a exigência principal do roteiro da trilha de Back-end.

Desta forma, foram priorizadas as funcionalidades essenciais para esse fluxo funcionar:

- Autenticação com JWT e perfis de acesso (ADMIN, GERENTE, CLIENTE);
- Cadastro de lojas e produtos;
- Criação de pedidos com canal obrigatório (APP, TOTEM, BALCAO, PICKUP ou WEB);
- Pagamento mock, com cenários de aprovação e negação;
- Atualização automática do status do pedido após o pagamento;
- Validações e padronização das respostas de erro;
- Documentação automática com Swagger/OpenAPI.

Outras funcionalidades foram projetadas, mas não foram incluídas nesta entrega, pois elas dependem de regras de negócio que não estão no escopo do fluxo crítico definido pelo roteiro da trilha de Back-end. São elas: controle detalhado de estoque, programa completo de fidelidade, emissão de relatórios para a matriz e cadastro de produtos sazonais com restrição de disponibilidade.

Essas funcionalidades podem ser adicionadas futuramente, visto que a arquitetura foi pensada para permitir esse crescimento sem precisar refazer o que já está pronto.

3 MODELAGEM E ARQUITETURA

Nesta seção são apresentados os artefatos de modelagem do sistema: o diagrama de casos de uso, o diagrama de classes, o diagrama entidade-relacionamento, a arquitetura em camadas e a descrição dos principais endpoints da API.

3.1 Diagrama de Casos de Uso

O diagrama de casos de uso apresenta os atores do sistema e as funcionalidades de cada um. Os atores identificados foram: Cliente, Atendente, Cozinha, Gerente/Administrador e o Sistema de Pagamento (ator externo).

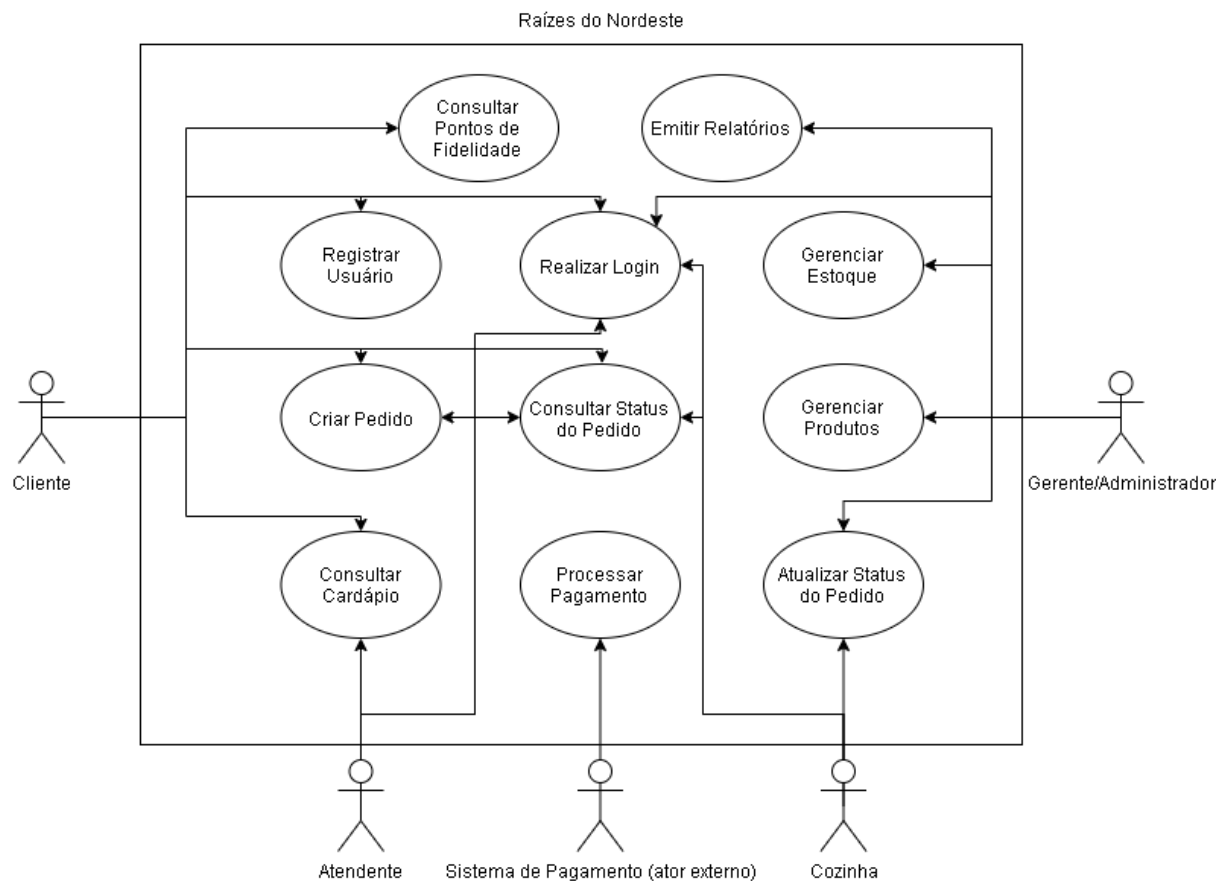


Figura 1 — Diagrama de Casos de Uso da rede Raízes do Nordeste.

3.2 Diagrama de Classes

O diagrama de classes representa as principais entidades do domínio do sistema e seus relacionamentos.

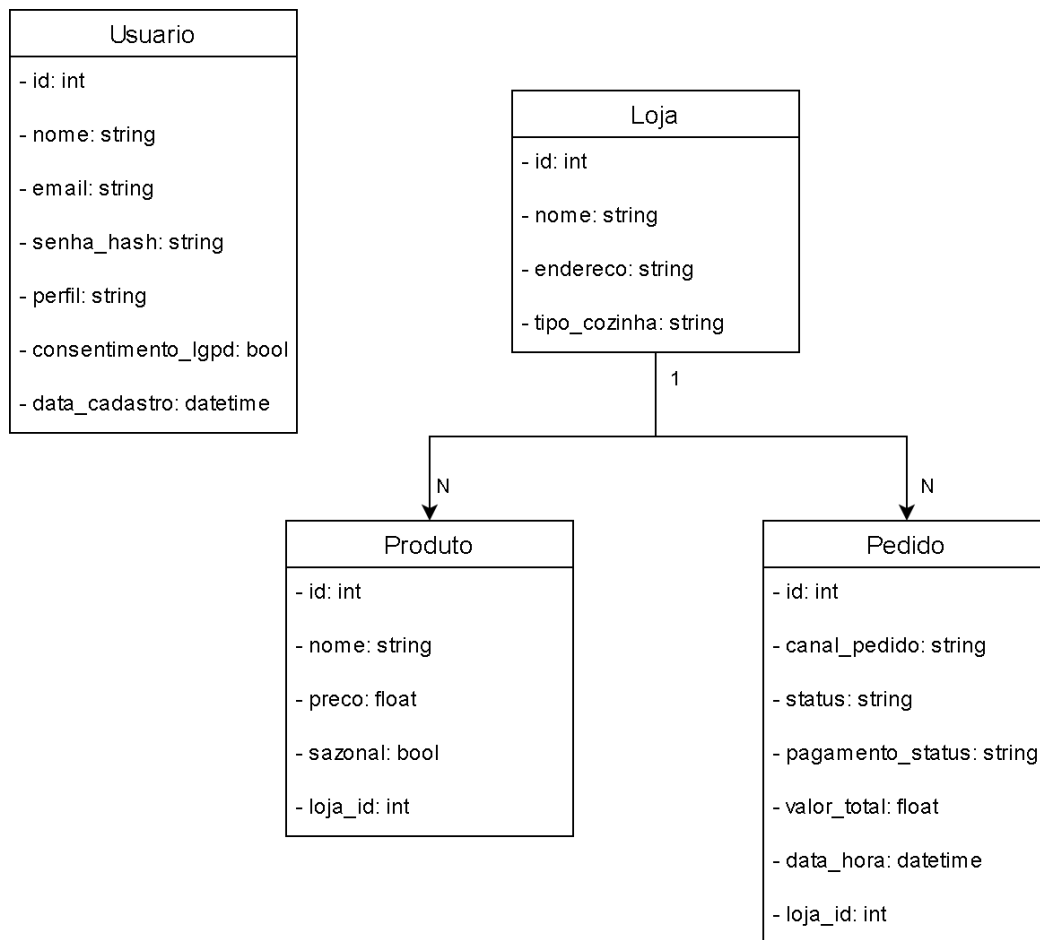


Figura 2 — Diagrama de Classes do Sistema.

As entidades são:

- **Usuario:** representa clientes, atendentes, cozinheiros e administradores do sistema. Possui perfis de acesso distintos;
- **Loja:** representa cada unidade da rede, com endereço e tipo de cozinha próprios;
- **Produto:** representa os itens do cardápio, vinculados a uma loja específica, com preço e flag de sazonalidade;
- **Pedido:** representa um pedido realizado em qualquer canal, com canal de origem, status e informações de pagamento.

3.3 Diagrama Entidade-Relacionamento (DER)

O DER apresenta a estrutura do banco de dados relacional utilizado pelo sistema (SQLite).

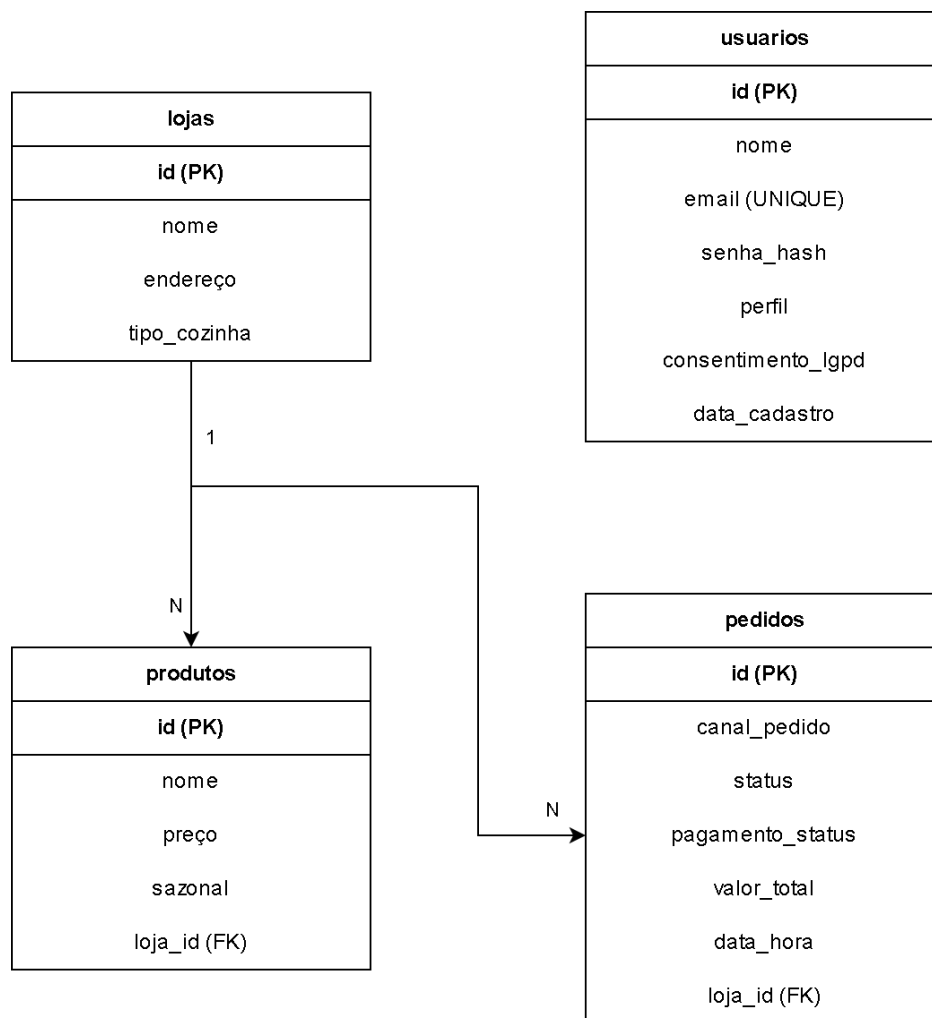


Figura 3 — Diagrama Entidade-Relacionamento do banco de dados.

As principais tabelas e relacionamentos são:

- usuarios: armazena dados de acesso, perfil e consentimento LGPD;
- lojas: armazena dados das unidades da rede;
- produtos: vinculada a lojas por chave estrangeira (loja_id);
- pedidos: vinculada a lojas por chave estrangeira (loja_id), armazena canal, status e valor.

3.4 Arquitetura em Camadas

A solução foi organizada em quatro camadas, garantindo separação clara de responsabilidades:

Camada	Responsabilidade	Exemplo no projeto
Domain	Entidades e regras de negócio	models.py (Usuario, Loja, Produto, Pedido)

Application	Casos de uso e orquestração	Serviços e lógica nos routers/
Infrastructure	Persistência e integrações	database.py (SQLAlchemy), pagamento mock
API	Rotas, autenticação, contratos	routers/ (auth, lojas, produtos, pedidos, pagamentos)

Essa organização segue princípios de Clean Architecture simplificada, permitindo que as regras de negócio não dependam de detalhes de infraestrutura.

3.5 Principais Endpoints da API

A API segue o padrão REST, com URLs no plural, uso de códigos HTTP coerentes e padronização de erros em JSON.

Método	Endpoint	Descrição	Autenticação
POST	/auth/registrar	Cadastro de usuário	Pública
POST	/auth/login	Autenticação e geração de token	Pública
GET	/auth/me	Dados do usuário logado	JWT
POST	/lojas/	Cadastro de loja	JWT (ADMIN/GERENTE)
GET	/lojas/	Listagem de lojas	Pública
POST	/produtos/	Cadastro de produto	JWT (ADMIN/GERENTE)
GET	/produtos/	Listagem de produtos	Pública
POST	/pedidos/	Criação de pedido	JWT
GET	/pedidos/	Listagem de pedidos (filtro por canal)	JWT

GET	/pedidos/{id}	Consulta de pedido por ID	JWT
POST	/pagamentos/{pedido_id}	Processamento de pagamento (mock)	JWT

3.6 Padrão de Erro

Todas as respostas de erro seguem o mesmo formato JSON:

- 200 OK — consulta bem-sucedida;
- 201 Created — recurso criado com sucesso;
- 401 Unauthorized — token ausente ou inválido;
- 403 Forbidden — usuário autenticado sem permissão;
- 404 Not Found — recurso não encontrado;
- 409 Conflict — conflito de regra de negócio;
- 422 Unprocessable Entity — dados inválidos.

3.7 Decisões Técnicas

Algumas decisões foram tomadas considerando o contexto do estudo de caso:

- FastAPI: escolhido pela geração automática de documentação Swagger/OpenAPI, tipagem forte com Pydantic e bom desempenho;
- SQLAlchemy + SQLite: ORM maduro com SQLite para simplificar a execução local, sem perder a possibilidade de migrar para PostgreSQL em produção;
- JWT: autenticação stateless, adequada para uma API consumida por múltiplos canais;
- Hash bcrypt: para armazenar senhas com segurança, em conformidade com a LGPD;
- Pagamento mock: simulação da integração com serviço externo, permitindo validar o fluxo sem dependência de provedores reais;
- Campo canal_pedido: obrigatório em todo pedido, garantindo a rastreabilidade multicanal exigida no estudo de caso;
- Não exposição de dados sensíveis: os schemas de resposta (Pydantic) não incluem o campo senha_hash. Isso garante que o hash de senha nunca trafegue pela API, em conformidade com a LGPD e com boas práticas de segurança.

4 ENTREGA TÉCNICA

Nesta seção são apresentados os artefatos entregues do projeto: o repositório público com o código-fonte, as instruções de execução, os endpoints implementados, as evidências de execução e a coleção de testes.

4.1 Repositório do Projeto

O código-fonte completo da API está disponível publicamente em:

<https://github.com/TarlanMarcos/raizes-nordeste-api>

O repositório contém:

- Código-fonte da API (Python + FastAPI);
- Estrutura organizada em camadas (domain, application, infrastructure e API);
- README com instruções de execução;
- Documentação Swagger/OpenAPI gerada automaticamente;
- Coleção Postman com todos os cenários de teste;
- Diagramas de modelagem (Casos de Uso, Classes e DER);

4.2 Como Executar a API

Para executar a API localmente, siga os passos do README:

4.2.1 Clone o repositório:

```
git clone https://github.com/TarlanMarcos/raizes-nordeste-api.git
cd raizes-nordeste-api
```

4.2.2 Crie e ative o ambiente virtual:

```
python -m venv .venv
.venv\Scripts\activate (Windows)
source .venv/bin/activate (Mac/Linux)
```

4.2.3 Instale as dependências:

```
Pip install "fastapi[standard]" sqlalchemy "python-jose[cryptography]" "passlib[bcrypt]" python-multipart
```

4.2.4 Execute a API:

```
uvicorn app.main:app --reload
```

4.2.5 Acesse a documentação Swagger:

<http://127.0.0.1:8000/docs>

4.3 Endpoints Implementados

A API possui os seguintes módulos de endpoints:

- /auth — registro, login e perfil do usuário;
- /lojas — cadastro e listagem de lojas;
- /produtos — cadastro e listagem de produtos;
- /pedidos — criação, consulta e listagem de pedidos com filtro por canal;
- /pagamentos — processamento de pagamento (mock).

4.4 Evidências de Execução

A seguir são apresentadas capturas de tela das principais operações da API, obtidas através do Swagger UI e do Postman.

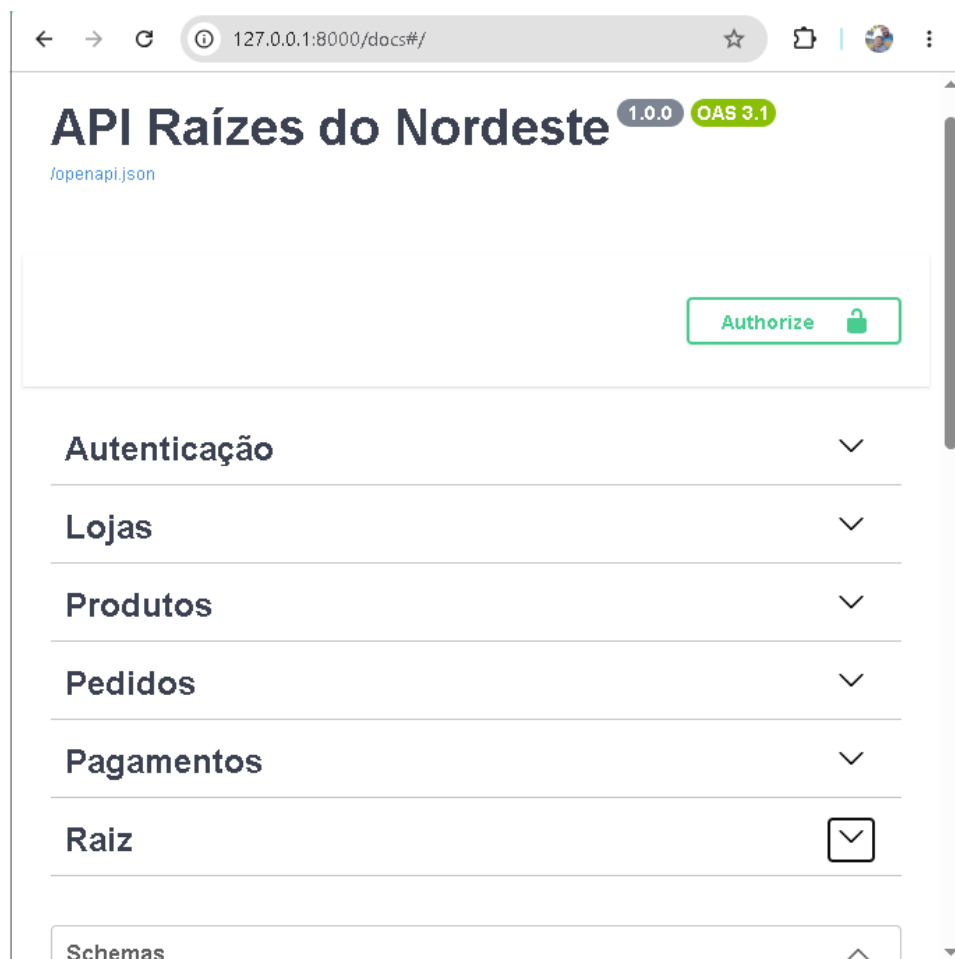


Figura 4 — Tela inicial do Swagger com todos os grupos de endpoints.

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/auth/regar' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nome": "Tarlan Admin",
    "email": "admin@raizes.com",
    "senha": "Senha@123",
    "perfil": "ADMIN",
    "consentimento_lgpd": true
  }'
```

Request URL

http://127.0.0.1:8000/auth/regar

Server response

Code	Details
201	Response body <pre>{ "id": 1, "nome": "Tarlan Admin", "email": "admin@raizes.com", "perfil": "ADMIN", "consentimento_lgpd": true, "data_cadastro": "2026-10-10T12:38:15.006554" }</pre>

Figura 5 — Cadastro de usuário com consentimento LGPD.

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "admin@raizes.com",
    "senha": "Senha@123"
  }'
```

Request URL

http://127.0.0.1:8000/auth/login

Server response

Code	Details
200	Response body <pre>{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbkByYW16ZAMuY29tIiwicGvZm1sIjoiaURNSU41LCJleHAiOjE3OTE2NTA1MDI9Lm00MgQWfG-rYasLt0YDUK1j8KBseuG4rR-4VSL6xU", "token_type": "bearer", "perfil": "ADMIN" }</pre>

Figura 6 — Login retornando token JWT.

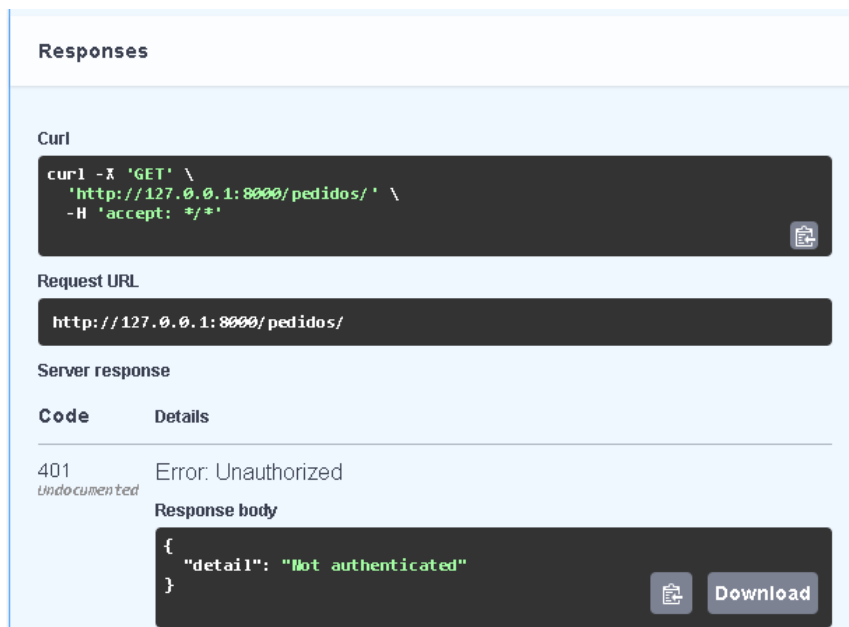


Figura 11 — Erro 401 ao acessar endpoint sem token.

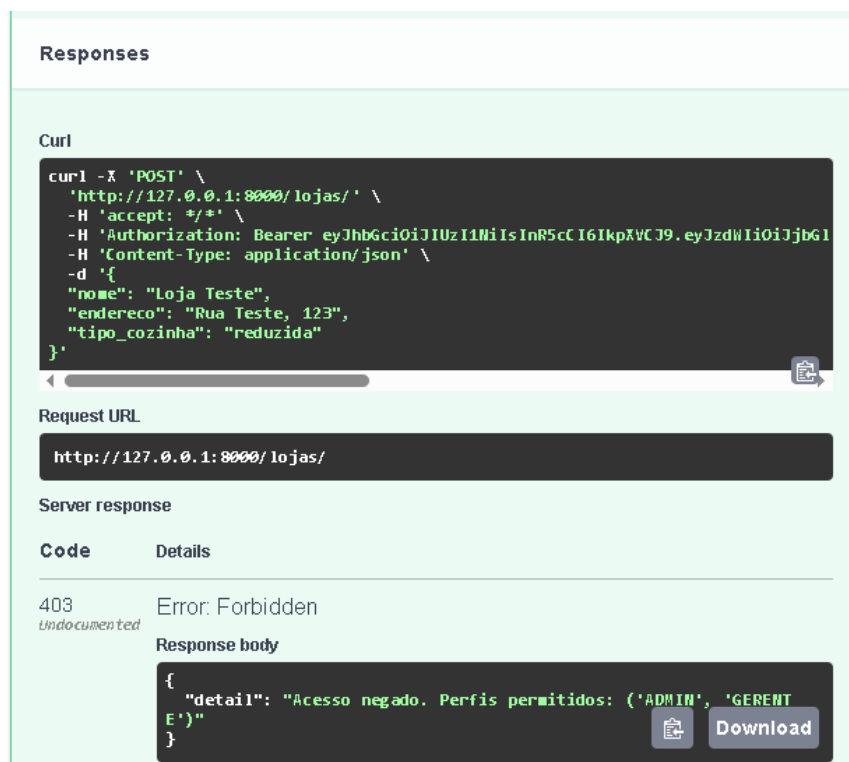


Figura 12 — Erro 403 ao tentar criar loja com perfil CLIENTE.

4.5 Coleção Postman

A coleção Postman com todos os cenários de teste está no arquivo `postman_collection.json`, disponível no repositório. Ela contém requisições organizadas por pasta (Auth, Lojas, Produtos, Pedidos, Pagamentos e Erros), o que permite reproduzir todos os cenários documentados na Seção 5 deste trabalho.

5 PLANO DE TESTES E EVIDÊNCIAS

Esta seção apresenta os testes aplicados na API. São descritos a estratégia adotada, os cenários executados e as evidências obtidas. O objetivo é mostrar que a solução funciona tanto nos fluxos esperados quanto nos casos de erro.

5.1 Estratégia de Testes

Os testes foram feitos de duas formas: pela interface do Swagger UI, que o próprio FastAPI gera automaticamente, e pela coleção Postman disponível no repositório. Foram criados cenários para os fluxos esperados e também para os casos de erro, cobrindo autenticação, autorização, validação de dados, regras do pedido, pagamento mock e padronização de erros.

5.2 Cenários de Teste

A seguir são apresentados os cenários de teste executados.

ID	Cenário	Endpoint	Entrada	Resultado esperado
T01	Login válido	POST /auth/login	E-mail e senha corretos	200 + token de acesso
T02	Login inválido	POST /auth/login	Senha incorreta	401 + "E-mail ou senha inválidos"
T03	Cadastro sem consentimento LGPD	POST /auth/registrar	consentimento_lgpd = false	422 + erro LGPD
T04	Criar loja como ADMIN	POST /lojas/	Dados válidos + token ADMIN	201 + loja criada
T05	Criar loja como CLIENTE	POST /lojas/	Token CLIENTE	403 + acesso negado
T06	Acesso sem token	GET /pedidos/	Sem token	401 + não autenticado
T07	Criar pedido com canal TOTEM	POST /pedidos/	canal_pedido = TOTEM	201 + pedido criado
T08	Criar pedido com loja inexistente	POST /pedidos/	loja_id = 999	404 + loja não encontrada
T09	Criar pedido com canal inválido	POST /pedidos/	canal_pedido = TELEFONE	422 + canal inválido
T10	Listar pedidos por canal	GET /pedidos/?canal_pedido=TOTEM	Filtro por canal	200 + lista filtrada
T11	Pagamento aprovado	POST /pagamentos/1	valor = 50,00	200 + status APROVADO
T12	Pagamento negado	POST /pagamentos/2	valor = 150,00	200 + status NEGADO
T13	Status atualizado após pagamento	GET /pedidos/1	Pedido já pago	200 + status PAGO

T14	Pagamento de pedido já pago	POST /pagamentos/1	Pedido com status PAGO	409 + pedido já pago
T15	Consultar pedido existente	GET /pedidos/1	—	200 + dados do pedido
T16	Consultar pedido inexistente	GET /pedidos/999	ID inexistente	404 + pedido não encontrado

5.3 Evidências

As evidências de execução estão organizadas no repositório do projeto:

- Coleção Postman: arquivo postman_collection.json;
- Prints das execuções: pasta prints/ (Swagger e Postman).

As figuras 4 a 12 na seção 4.4, apresentam as capturas de tela das principais execuções.

5.4 Cobertura dos Testes

Os cenários cobrem os seguintes aspectos exigidos pelo roteiro:

- Autenticação e autorização (T01, T02, T05, T06);
- Validação de dados (T03, T08, T09);
- Regras de negócio do fluxo principal (T07, T10, T15, T16);
- Pagamento mock e atualização de status (T11, T12, T13, T14);
- Padronização de erros (401, 403, 404, 409, 422).

6 CONCLUSÃO

Desenvolver este projeto foi uma oportunidade de aplicar, na prática, os conhecimentos de Engenharia de Software vistos ao longo do curso. O estudo de caso da rede "Raízes do Nordeste" trouxe um cenário realista, com ambiguidades e prioridades que precisaram ser decididas e justificadas.

O principal aprendizado foi perceber que um profissional desta área vai muito além do código. A modelagem do domínio, a definição dos contratos da API, a padronização dos erros e os cuidados com segurança e privacidade têm o mesmo peso que a implementação em si. Uma das grandes dificuldades foi manter a coerência entre o DER, o Diagrama de Classes e os endpoints implementados.

O pagamento foi tratado como um serviço externo desacoplado, conforme propõe o estudo de caso. A API apenas solicita o processamento, recebe a confirmação ou negativa e atualiza o status do pedido. Essa escolha reduz riscos e facilita a troca por um provedor real no futuro. Foram testados cenários de aprovação e negação, com regras claras e respostas padronizadas.

Em relação à LGPD, foram adotados cuidados básicos, mas consistentes: senhas armazenadas com hash bcrypt, consentimento explícito no cadastro, autenticação via JWT, controle de acesso por perfil e nenhuma exposição de dados sensíveis nas respostas.

O plano de testes cobre cenários de sucesso e de erro, incluindo autenticação, autorização, validações, regras de negócio e integração com pagamento. A coleção Postman disponível no repositório permite que qualquer pessoa reproduza os testes, o que dá autonomia para validar a API.

Como evolução futura, ficam algumas funcionalidades que não entraram nesta entrega: controle completo de estoque com movimentações de entrada e saída, programa de fidelidade com acúmulo e resgate de pontos, relatórios consolidados para a matriz, cadastro de produtos sazonais com restrição de disponibilidade e logs de auditoria para operações sensíveis. A base construída permite adicionar tudo isso sem precisar reestruturar o que já existe.

Por fim, este projeto não é apenas uma entrega acadêmica, mas um exercício de postura profissional, porque buscou simular decisões e desafios que aparecem no dia a dia do desenvolvimento de software.

7 REFERÊNCIAS

FASTAPI. Documentação oficial. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: out. 2026.

SQLALCHEMY. Documentação oficial. Disponível em: <https://docs.sqlalchemy.org/>. Acesso em: out. 2026.

PYDANTIC. Documentação oficial. Disponível em: <https://docs.pydantic.dev/>. Acesso em: out. 2026.

JWT. JSON Web Tokens. Disponível em: <https://jwt.io/>. Acesso em: out. 2026.

OWASP. Password Storage Cheat Sheet. Disponível em: <https://cheatsheetseries.owasp.org/>. Acesso em: out. 2026.

BRASIL. Lei Geral de Proteção de Dados Pessoais (LGPD). Lei nº 13.709, de 14 de agosto de 2018. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: out. 2026.

SOMMERVILLE, I. Engenharia de Software. 10. ed. São Paulo: Pearson, 2018.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de Software: Uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

FOWLER, M. UML Essencial: um breve guia para linguagem padrão. 3. ed. Porto Alegre: Bookman, 2005.

8 DECLARAÇÃO DE USO DE IA

Durante o desenvolvimento deste trabalho, foram utilizadas ferramentas de Inteligência Artificial (IA) como apoio em algumas etapas. A ferramenta usada foi o Claude, da Anthropic.

A IA foi utilizada para:

- Entender melhor alguns conceitos técnicos, como JWT, SQLAlchemy, FastAPI e LGPD;
- Organizar e estruturar o documento;
- Auxiliar no desenvolvimento de códigos;
- Resolver erros de implementação durante o desenvolvimento da API.

Todo o conteúdo final foi revisado e adaptado pelo autor, que assume total responsabilidade pelo trabalho apresentado.

9 INFORMAÇÕES DE ENTREGA

- Repositório do projeto: <https://github.com/TarlanMarcos/raizes-nordeste-api>;
- Evidências de autoria (commits): <https://github.com/TarlanMarcos/raizes-nordeste-api/commits/main>;
- Documentação Swagger: <http://127.0.0.1:8000/docs>
- Conteúdo do repositório: Código-fonte da API (Python + FastAPI);
- README com instruções de execução;
- Coleção Postman com os 16 cenários de teste;
- Diagramas de modelagem (Casos de Uso, Classes e DER);
- Prints das execuções (Swagger);
- PDF do documento do projeto;
- Declaração de uso de IA: Disponível na seção 8 deste documento.

Aluno: Tarlan Marcos Dalla Vecchia

RU: 4896093

CST em Análise e Desenvolvimento de Sistemas